

Sprint 2D — Corrected Roadmap + Execution Graph Grammar Design

Supersedes the “just do 2D.5” plan. Written after the Rule 1 audit
(`SPRINT_2D5_RULE1_EQUIVALENCE_AUDIT.md`) and the coverage baseline from
`tools/measure-graph-coverage.ts` .

The reframing (agreed)

The audit proved Rule 1 fails: only **4 of 79** scenarios run from the graph.
The fix is **not** a different deletion strategy — it is finishing the graph.
Adding action/assertion templates is **Execution Graph completion**, i.e.
finishing 2D.3 / 2D.4 — it is **not** deletion work and must not be called 2D.5.

The deeper diagnosis: the blocker is **Execution Graph expressiveness**, not
Script Generation. The graph must become a small **language** (composable
primitives), not a growing catalog of bespoke scenario fields.

Objective gate (new, quantitative)

`tools/measure-graph-coverage.ts` computes, from the same KB the engine reads:

$$\text{Execution Graph Coverage} = \frac{\text{scenarios with BOTH actions[] AND assertions[]}}{\text{total scenarios}}$$

A scenario counts only when it owns **both** halves — because the `actions[]` and
`assertions[]` gates in `script-gen-engine.ts` are independent, so owning one but
not the other still drops the other half to legacy inference.

Baseline today (`npx ts-node tools/measure-graph-coverage.ts` , exit 1 = gate closed)

module	owned	coverage
authentication	4/13	31%
crud	0/9	0%
search	0/8	0%
checkout	0/9	0%
payment	0/8	0%
admin	0/6	0%
workflow	0/7	0%
reporting	0/6	0%
import	0/7	0%
export	0/6	0%
OVERALL	4/79	5%

2D.5 (deletion) may begin only when OVERALL = 100%. The tool exits non-zero until then, so it is a CI gate, not a judgment call.

Corrected roadmap

Every sprint now has one measurable outcome (a jump in graph coverage), and deletion is quarantined to the very end.

Sprint	Outcome	Coverage target	Deletion?
2D.3A	Authentication actions complete (all 13)	auth actions 100%	none
2D.4A	Authentication assertions complete (all 13)	auth 13/13	none
2D.3B	CRUD actions+assertions	crud 100%	none
2D.3C	Search actions+assertions	search 100%	none
2D.3D	Checkout actions+assertions	checkout 100%	none
2D.3E...	payment / admin / workflow / reporting / import / export	each →100%	none
2D.5	Delete legacy (tc-StepsToCode , buildTcAssertions , ScenarioIntelligence , buckets, regex) + permanent architecture test	overall stays 100%, goldens byte-identical	all

Phase A = everything above 2D.5. Phase B = 2D.5 only.

Grammar design — the graph becomes a small language

Principle: add composable primitives, never per-scenario fields. Wrong:

repeatTimes , browserRestart , sessionRefresh , whitespacePassword , sqlInjectionUser , loginLogoutReloginAction . Right: a handful of primitives that compose into every scenario.

First: re-triage the 9 “pending” auth scenarios against the POST-2D.4 grammar

The pending reasons were authored **before** 2D.4 assertions + afterAction landed (one literally says “until 2D.4 assertions land”). The coverage meter shows all 9 already own assertions (0 / 9 / 0 gap = assertions-only). So the real remaining question is only **which need a NEW action primitive**:

Pending scenario	Needs new grammar?	Composes from
auth-neg-locked-user	No (re-triage)	same actions as auth-pos-valid + locked_account dataset role + existing error assertion
auth-edge-password-masking	No (re-triage)	fill(password) + existing attribute assertion (type=password)
auth-pos-logout	No (re-triage)	fill , fill , click(login) , click(logout) , igate(protected) + assertions ordered by afterAction
auth-edge-whitespace-case	transform	transform(username, pad) → fill → click → as- sert(error)
auth-neg-invalid-identifier-format	transform	transform(username, mal- form) → fill → click → assert(error)
auth-sec-injection	transform	transform(username, sql_payload) → fill → click → assert(error)
auth-sec-logout-threshold	repeat	repeat(count:N)[fill, fill, click] → as- sert(locked)
auth-pos-remember-me	restartBrowser	fill , fill , check(remember_me) , click , re- startBrowser , navigate → assert(logged in)
auth-sec-session	fresh context	restartBrowser (clean session) → navig- ate(protected) → as- sert(redirected to login)

⇒ **The entire pending set reduces to exactly THREE new primitives**

(transform , repeat , restartBrowser), and **3 of 9 need nothing new** — they were blocked only because assertions didn't exist yet. This must be **verified during authoring**, but it shows the grammar surface is tiny.

The three primitives (minimal specs)

1. transform — derive a mutated value (covers whitespace / malform / injection / max-length / case).

The reusable part is the transform verb vocabulary, not per-case fields. Two framings — pick one (see decision below):

- As a value-expression (recommended — no new node type, no inter-node state):

```
jsonc
{ "action": "fill", "target": "username",
  "value": { "source": "@dataset.username", "transform": "pad_whitespace" } }
```

- As a standalone node (matches the “transform → fill” sequence mental model, but introduces a derived-value slot between nodes):

```
jsonc
{ "action": "transform", "target": "username", "transform": "pad_whitespace", "from":
"@dataset.username" }
{ "action": "fill", "target": "username", "value": "@transformed.username" }
Transform verbs (frozen small set): trim | pad_whitespace | uppercase |
lowercase | prepend:<s> | append:<s> | sql_payload | max_length:<n> | malform.
```

2. repeat — run a nested block N times (covers lockout threshold).

```
jsonc
{ "action": "repeat", "count": 5, "steps": [
  { "action": "fill", "target": "username", "value": "@dataset.username" },
  { "action": "fill", "target": "password", "value": "@dataset.wrong_password" },
  { "action": "click", "target": "login_button" }
] }
```

One structural addition: an action may nest `steps[]`. The emitter loops; still zero inference.

3. restartBrowser — end the session / start a clean context (covers remember-me and session).

```
jsonc
{ "action": "restartBrowser" } // no target; optional { "keepStorage": true } for remember-me
```

Remember-me keeps storage; session-access uses a clean context (no storage).

Explicitly NOT adding

- No request primitive — `auth-sec-session` composes from `restartBrowser` (clean context) + `navigate(protected)` + a url assertion.
- No logout / login compound actions — they are just `click` sequences.
- No per-scenario action types of any kind.

The one design decision I need from you

Transform as a value-expression (recommended) vs. as a standalone node.

Value-expression keeps the grammar flatter (only `repeat` adds nesting) and avoids an inter-node “derived variable” concept; the standalone-node form matches your “transform → fill” sequence sketch more literally. Everything else above is, I believe, uncontroversial given your direction.

Consciously re-opening, then re-freezing the grammar

You froze the grammar at the end of 2D.4. This design **re-opens it once, minimally** (3 primitives + optional value-expression + `steps[]` nesting), to make the graph expressive enough to be the single source of truth — which is the precondition for the freeze to actually hold. After Phase A authoring completes and coverage hits 100%, the grammar **re-freezes for good** and every later sprint answers only “did the generated artifacts get better?”

Definition of Done

Phase A (graph completion) — per module

- [] every scenario owns `actions[]` **and** `assertions[]`
- [] `measure-graph-coverage.ts` shows the module at 100%
- [] no new bespoke fields — only the 3 primitives / value-expressions
- [] all existing golden + fidelity tests still pass, byte-identical

Phase A exit (whole graph)

- [] `measure-graph-coverage.ts` OVERALL = 100% (exit 0, gate open)
- [] grammar re-frozen (contract doc updated + version bumped)

Phase B (2D.5 deletion) — only after the gate opens

- [] delete `tcStepsToCode`, `buildTcAssertions`, `ScenarioIntelligence`, `buckets`, `regex`
- [] add the permanent architecture test (Script Gen may import only Scenario Graph / Execution Graph / Emitter / Locator Resolver / Dataset Resolver)
- [] goldens byte-identical; Script Gen line count drops (deletions > additions)